

libmr — Relazione di progetto

Laboratorio 2 A, a.a. 2025-26

Fabio Tognarelli, 693132

Indice

1. Architettura generale	1
2. Interfaccia pubblica	2
2.1 Gestione degli Errori	3
3. Processi	5
4. Thread C11	5
5. Code interne	6
6. Protocollo sulle pipe	6
7. Raggruppamento per token	8
8. Formato e ordine dell'output	8
9. File di log	9
10. Test	10

1. Architettura generale

In questa sezione iniziale viene presentato il progetto, compreso della sua architettura e dei processi principali.

Partiamo con il dire che si tratta di un framework utilizzabile tramite la libreria statica `libmr(libmr.a)`, con header pubblico `include/mr.h`.

Il programma utente dovrà fornire in input solo due funzioni di callback, relative al processo di mapping e di reducing (`mr_mapper_t`, `mr_reducer_t`); una volta fatto ciò il framework leggerà l'input, eseguirà la pipeline di processi, raggrupperà per token e scriverà l'output risultante.

Vi sono **3 processi** principali, i quali comunicano via **pipe**

- **main** (processo chiamante): legge i file di input, serializza le righe verso il mapper, raccoglie i record dal reducer, li ordina e li scrive su un file di output;
- **mapper**: deserializza le righe, invoca la callback mapper su thread C11, serializza le coppie `<token, valore>` verso il reducer;
- **reducer**: accumula tutte le coppie, raggruppa per token, invoca la callback reducer una volta per token distinto, serializza i risultati verso il main.

Input:

- *file regolare* singolo,

- *directory* contenente più file di testo (non è prevista esplorazione ricorsiva delle sottocartelle, si considerano quindi solo i file direttamente figli della cartella stessa), in **ordine lessicografico** sul nome.

Ogni file viene trattato come una sequenza di **righe logiche**: il carattere di fine riga: '\n' non fa parte del contenuto passato al mapper, ultima riga inclusa. File vuoti e righe vuote sono ammessi e gestiti correttamente. Per quanto riguarda la serializzazione della riga sulla pipe nel dettaglio consultare la sezione 6.

I puntatori in `mr_file_line_t` e in `mr_value_t` valgono solo durante la callback: emit copia token e byte opachi prima di ritornare.

Il *seguente grafico* mostra la **pipeline end-to-end** dei processi nel programma. La pipeline è lineare: il main è sia sorgente delle righe sia il collettore dei risultati. Sotto ogni stadio vi è l'ordine in cui si chiude il lato *write* (EOF sul *read* da main -> mapper).

Senza i punti (1)–(3) un lettore resta bloccato in attesa di ricevere qualcosa in input. Gli worker paralleli gestiti dai thread non chiudono la pipe verso il reducer o verso il main.

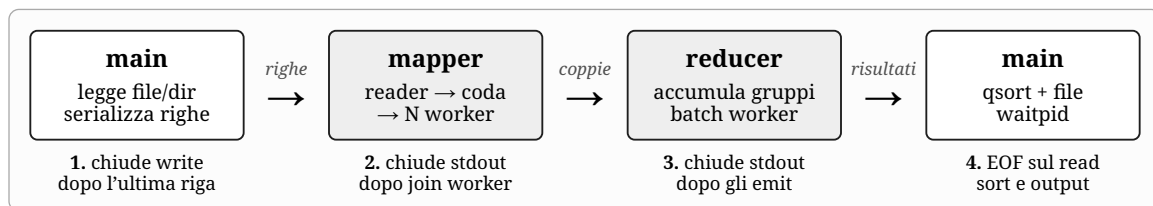


Figure 1: Pipeline di processi e chiusura delle pipe.

2. Interfaccia pubblica

Contratto pubblico di `mr.h` utilizzabile lato utente:

- `mr_t`: tipo opaco di una variabile `mr`
- `mr_attr_t`: contiene i valori degli attributi di `mr`. Si tratta di un valore copiabile; dopo `mr_create` il chiamante può eliminarlo o modificarlo senza effetto sull'istanza attuale.
- `mr_attr_init`: inizializza `attr` con valori di default
- `attr_setters`: lista di funzioni per settare ogni valore di `attr`
- `mr_create`: crea un `mr`
- `mr_start`: esecuzione di un `mr` (bloccante)
- `mr_destroy`: elimina un `mr`
- Valori Default di `mr_attr_init`: `mapper_threads = 1`, `reducer_threads = 1`, `queue_size = 64`, `log_file = NULL` (file `mr.log`). I setter rifiutano 0 per thread e coda.

- `mr_create` copia configurazione, puntatori alle callback e `user_arg`. `user_arg` non è interpretato; dopo `fork` le modifiche nei figli non sono visibili al padre né all'altro figlio.
- Valori intermedi e risultati: byte opachi (`data + size`). `size == 0` ammette `data == NULL`. Il framework non usa `strlen/strcmp/printf("%s")` su di essi. Il token è l'unica stringa C interpretata (alfanumerica ASCII, `'\0'` finale; il `'\0'` non sta sulla pipe).
- Ritorno: 0 successo, -1 errore, `errno` ove appropriato.

2.1 Gestione degli Errori

Verso il programma utente il contratto è semplice: ogni funzione pubblica restituisce 0 in caso di successo e -1 in caso di errore, con `errno` impostato. Chi chiama `mr_start` può quindi limitarsi a un `perror` e, se serve il dettaglio, leggere il file di log. Dietro questo contratto la gestione non può essere uniforme. Un errore può nascere nel processo principale, nel mapper o nel reducer; `errno` non attraversa il `fork`; una `close` o una scrittura sul log sovrascrivono `errno` della failure originale; un descrittore lasciato aperto nel processo sbagliato impedisce l'EOF e blocca la pipeline.

Per questo il framework distingue **chi può solo fallire** e **chi può parlare**.

Le funzioni di trasferimento e della coda (`io.c`, `queue.c`) sono silenziose, ovvero impostano `errno` e ritornano -1, senza scrivere sul log e senza `perror`. Un header con lunghezza negativa, nulla (per il token) o oltre i tetti della sezione 6 vale `EINVAL`; una `malloc` fallita vale `ENOMEM`; una `read` o `write` fallita lascia l'`errno` del kernel. L'EOF sulla pipe non è un errore: si segnala con `EOF_REACHED` e `errno` resta intatto. Queste funzioni girano sia nel main sia nei figli. Se loggassero, la stessa failure apparirebbe due volte, e i test sul protocollo (`tests/io.c`) non potrebbero verificare `EINVAL` in isolamento.

Mapper, reducer e la lettura dell'input conoscono il contesto, quindi loggano. Nei processi figli un flag `ctx.error` ricorda che qualcosa è andato storto. I **thread worker non abortiscono da soli la pipeline**: segnano l'errore, finiscono il lavoro già in coda, poi il processo chiude `stdout` — così il lettore a valle riceve EOF — e ritorna -1. Se fallisce `mr_send_input`, il lato write verso il mapper **non** viene chiuso in `input.c`: quella chiusura spetta al parent. Un `close` in due posti diversi rischia un doppio `close` oppure un EOF prematuro, con il mapper che crede finito l'input mentre il main è ancora in errore.

Subito dopo il `fork` un fallimento di `dup2` o `close` non può tornare a `mr_start` come una funzione C normale: il figlio è già un processo distinto. In quel caso si usa `mr_child_fail`: una riga sul log e `_exit(1)`. Lo status 1 è l'unico canale con cui il figlio comunica il fallimento al padre.

Nel parent, una volta create le pipe (e, dopo i `fork`, i figli), lo stato ancora vivo sta in `mr_start_state_t`: descrittori aperti, record accumulati, PID, se i figli esistono. Ogni path di errore salva `errno` **prima** di loggare o chiudere, poi chiama `mr_start_cleanup`:

si tratta appunto di una funzione di cleanup, la quale chiude ciò che è ancora aperto, libera i record e, se i figli sono già stati creati, li attende con `waitpid`. Prima dei `fork` il cleanup resta manuale: non ha senso fare `waitpid` su processi che non esistono. Un figlio uscito con codice diverso da 0, o ucciso da un segnale, diventa `errno = EIO` per il chiamante. Il parent non rilogga la stessa failure se il figlio è già uscito con 1 — l’ha già scritta sul log. Logga invece i segnali (`WIFSIGNALED`), perché in quel caso il figlio può non essere riuscito a scrivere.

Un messaggio troncato o una lunghezza invalida, **dentro** `io.c`, è `EINVAL`. Se accade nel mapper o nel reducer, il figlio termina con `_exit(1)` e `mr_start` restituisce `EIO`. Lo stesso `EINVAL` visto dal programma utente vale per argomenti sbagliati, path di input non valido, o un secondo `mr_start` sullo stesso handle: cioè per errori che non hanno avviato, o non devono riavviare, la pipeline. Una pipe o un `fork` falliti **nel main** invece arrivano al chiamante con l’`errno` del kernel.

Errore	Quando	Effetto
EINVAL	Puntatori NULL; thread o <code>queue_size</code> pari a 0; path di input non valido; secondo <code>mr_start</code> sullo stesso handle. In <code>io.c</code> : lunghezze negative o oltre i tetti.	Ritorno -1, <code>errno = EINVAL</code> . Se l’errore è nel figlio, il chiamante di <code>mr_start</code> vede <code>EIO</code> .
ENOMEM	Fallimento di <code>malloc</code> , <code>realloc</code> o <code>calloc</code> .	Ritorno -1 con <code>errno</code> del sistema. Cleanup di quanto già allocato.
syscall nel main	Fallimento di <code>pipe</code> , <code>fork</code> , <code>close</code> , <code>open</code> , <code>waitpid</code> nel processo principale.	Ritorno -1 con l’ <code>errno</code> del kernel. Log e <code>mr_start_cleanup</code> (chiusura FD, <code>waitpid</code> sui figli già creati).
EIO	Figlio uscito con codice diverso da 0 (<code>mr_child_fail</code> o <code>ctx.error</code>) oppure terminato da un segnale.	Il figlio ha già loggato (salvo i segnali). Il main in <code>waitpid</code> imposta <code>errno = EIO</code> .

Scelta. `io.c` e `queue.c` non loggano. Il parent usa uno snapshot (`mr_start_state_t`) e un cleanup unico; i figli in setup usano `mr_child_fail`. Si salva `errno` prima di ogni `log/close/waitpid`.

Perché. Lo stesso I/O vive in tre processi: un log in foglia produce righe doppie. Un cleanup sparso lascia descrittori aperti e la pipeline si blocca in attesa di EOF. Dopo una `close`, `errno` non è più quello della failure. `_exit(1)` è l’unico canale figlio-padre; il parent lo traduce in `EIO` e non ripete il messaggio già scritto dal figlio.

Scelta. Dopo un errore in `mr_start`, l’handle `mr_t` non è riutilizzabile: occorre `mr_destroy` e un nuovo `mr_create` prima di un altro `mr_start`.

Perché. `mr_start` crea processi, pipe e stato interno (record accumulati, FD). Un secondo avvio sullo stesso handle mescolerebbe descrittori e PID. Il contratto one-shot evita un reset parziale difficile da rendere corretto.

3. Processi

Qui vengono descritti i processi e tutte le chiamate di sistema che vengono effettuate su di essi. Una volta che viene lanciato `mr_start`:

1. Creazione di tre pipe: `main_to_mapper`, `mapper_to_reducer`, `reducer_to_main`;
2. `fork` del mapper; nel figlio `dup2` su `stdin/stdout`, chiusura di **tutti** i FD delle tre pipe (restano `stdin/stdout` già ridiretti);
3. `fork` del reducer; stesso schema (`stdin` ← `read` di `mapper_to_reducer`, `stdout` → `write` di `reducer_to_main`);
4. nel main restano aperti solo il `write` verso il mapper e il `read` dal reducer; gli altri sei estremi si chiudono;
5. invio righe, chiusura del `write` verso il mapper, raccolta risultati fino a EOF, `qsort`, scrittura file, `waitpid` di mapper e reducer.

I figli eseguono `mapper_process_main` / `reducer_process_main` e poi `_exit`. Non c'è nessuna `exec`. I `fork` avvengono **prima** di qualsiasi `thrd_create` (nel main non ci sono thread C11 del framework, sono presenti solo in mapper e reducer).

Se il `fork` del reducer fallisce, il main chiude le pipe e fa `waitpid` del mapper già avviato. I figli in errore di `dup2/close` usano `_exit` dopo aver scritto sul log.

4. Thread C11

Header `<threads.h>`. Niente `pthread_*`. Firma dei thread: `int (*)(void *)`.

Mapper. Un thread *reader* legge da `stdin` i messaggi riga, li inserisce nella coda (sezione 5) e alla EOF chiama `mr_queue_close`. `N = mapper_threads` worker fanno pop, ricostruiscono `mr_file_line_t` locale, invocano la callback del mapper, emettono coppie. La scrittura su `stdout` è gestita da una `mtx_t`: un messaggio logico non deve mescolarsi con un altro e viene quindi sincronizzato. In `emit` viene fatta la validazione del token (non vuoto, solo caratteri alfanumerici ASCII A-Z, a-z, 0-9). Chiusura `stdout`: dopo `thrd_join` di reader e worker (passo 2 in figura 1). Id di log: `reader = 0`, `worker = 1...N`.

Reducer. Allo stesso modo del mapper, un thread *reader* legge coppie da `stdin` e le inserisce nei gruppi (sezione 7). I worker **non** partono in parallelo al reader: si fa `thrd_join` del reader (EOF = nessuna coppia ulteriore), poi si invoca la callback reducer su ogni gruppo per token.

Dopo il join del reader, i gruppi si elaborano a **batch** di al più `reducer_threads`: per ogni batch si creano i worker (un gruppo a testa), poi si fa join e si passa al batch successivo.

Questo perché. La specifica del progetto impone il reduce solo a gruppi completi: non c'è produzione concorrente di gruppi durante il reduce, quindi non

serve una coda produttore-consumatore in questa fase. Il batch usa il parametro `reducer_threads` senza tenere thread idle durante tutta la lettura. Anche qui viene usata una mutex su `stdout`: più worker dello stesso batch possono fare `emit` insieme ed è un'eventualità che va evitata.

5. Code interne

Si tratta di una coda interna ai processi, utilizzata solo nel **processo mapper**, per coordinare reader e worker. Non è una pipe e non è condivisa fra processi, in quanto come già detto rimane interna. Capacità = `queue_size` (numero di elementi, non byte). Buffer circolare di puntatori a righe allocate in heap.

Se la coda è piena, il reader attende `not_full`; se invece è vuota, il worker attende `not_empty`; `close` sblocca i consumer e i pop successivi falliscono (fine lavoro).

```
typedef struct {
    void **items;
    size_t cap, head, tail, count;
    mtx_t mtx;
    cnd_t not_full, not_empty;
    int closed;
} mr_queue_t;
```

Come tipo di coda è stata scelta una **bounded queue**: in questo modo il `push` si blocca a coda piena; `queue_size` è il numero massimo di righe in volo nel mapper.

Il tetto esiste perché i worker possono essere più lenti del reader: senza un limite, ogni riga già letta da `stdin` resterebbe in heap in attesa, e su un file grande la memoria del mapper crescerebbe senza controllo. Con `queue_size` il `push` si ferma a coda piena, il reader smette di svuotare la pipe e il main a monte rallenta: restano in volo al più tante righe quante l'utente ha chiesto, e il parametro ha un effetto concreto sulla memoria e sul ritmo della pipeline.

Il reducer non usa questa coda: l'accumulo è la tabella dei gruppi (sezione 7).

6. Protocollo sulle pipe

Le **tre pipe** della pipeline trasportano dati eterogenei: righe di testo, coppie `<token, valore>` e risultati finali. Poiché `processed_token` e i risultati del reducer sono sequenze opache — possono contenere byte nulli e non sono stringhe C — scrivere testo libero sul descrittore non basterebbe. Ogni messaggio è quindi **length-prefixed**: è accompagnato da un header con lunghezze esplicite, seguito dai byte effettivi.

Una singola `read` o `write` non trasferisce necessariamente un intero messaggio logico. Le funzioni `readn` e `writen` ripetono la syscall fino a ottenere esattamente `n`

byte desiderati, gestendo `EINTR` e i trasferimenti parziali. La fine del flusso non è un messaggio speciale: si segnala chiudendo il lato di scrittura della pipe, così il lettore a valle riceve EOF.

Le **lunghezze negli header** sono di tipo `int`, come richiesto dalla specifica. Prima di convertirle a `size_t` — e prima di usarle per allocare — il framework le valida: si rifiutano i valori negativi e quelli che eccedono i tetti prefissati (paragrafo successivo). Un `token_len` pari a 0 è invalido, ricordando che esso è una sequenza alfanumerica non vuota. Il token viaggia sulla pipe senza il terminatore `'\0'`; il ricevente alloca `token_len + 1` byte e lo aggiunge localmente, così da poterlo trattare come stringa C. I payload opachi occupano esattamente `value_len` byte: nessun terminatore implicito, nessuna interpretazione del contenuto.

Qui elencati i tetti massimi per token e value:

- `MR_MAX_TOKEN_LEN` = 1 MiB
- `MR_MAX_VALUE_LEN` = 64 MiB
- `MR_MAX_LINE_LEN` = 64 MiB
- `MR_MAX_NAME_LEN` = 4096.

Il testo chiede **limiti ragionevoli documentati**, questi valori sono stati ritenuti i più consoni.

Il messaggio **riga** (`main` → `mapper`) porta, oltre al contenuto, il **contesto** che la **callback mapper** deve vedere in `mr_file_line_t`: nome del file e numero di riga. L'header è seguito dai byte del nome e da quelli della riga; il `'\n'` di fine riga non sta sul payload.

Di seguito la struttura che rappresenta header di una linea in entrata nel mapper

```
typedef struct {
    int file_name_len;
    int line_len;
    unsigned long line_number;
} mr_line_header_t;
/* poi: file_name_len byte di nome, line_len byte di riga (senza '\n') */
```

Il messaggio **coppia** (`mapper` → `reducer`) e il messaggio **risultato** (`reducer` → `main`) condividono lo **stesso header**. Nel secondo caso il campo valore contiene i byte opachi emessi dal reducer, non un `processed_token`. Se `value_len` vale 0, non c'è payload e il ricevente passa `NULL`.

Di seguito la struttura che rappresenta header di una coppia in entrata nel reducer

```
typedef struct {
    int token_len;
    int value_len;
```

```

} mr_pair_header_t;
/* poi: token_len byte di token, value_len byte opachi */

```

7. Raggruppamento per token

Nel reader del reducer, ogni coppia entra in `add_or_create_group`. Un gruppo corrisponde ad un token distinto; la callback reducer parte **una volta** per gruppo, con tutti i valori, solo dopo EOF.

Di seguito la struttura che rappresenta il gruppo di un token distinto

```

typedef struct {
    char *token;
    void **data;      /* copia heap di ogni processed_token */
    size_t *sizes;
    size_t count;
    size_t cap;
} token_group_t;

```

Accodamento. Lookup: scansione lineare con `strcmp` sul token (stringa C già terminata in ricezione). Se esiste, si accoda una **copia** dei byte; se `value_size == 0`, si accoda `NULL`. Se non esiste, si fa `realloc` dell'array di `token_group_t` a `groups_len + 1` (un gruppo in più). I buffer `data/sizes` del nuovo gruppo partono da `cap = 1` e raddoppiano quando `count == cap`. **Dopo EOF:** `qsort` dei gruppi per token (`strcmp`), poi i batch della sezione 4. L'ordine sulla pipe verso il main **non** è il contratto di determinismo (i worker di uno stesso batch emettono in concorrenza): l'ordine del file viene trattato nella sezione 8.

Tabella = **array dinamico** di `token_group_t` con lookup lineare; ordinamento dei gruppi con `qsort` dopo EOF. Il reduce parte solo a input esaurito: non serve una struttura concorrente durante l'inserimento. Il token è una stringa C, quindi `strcmp` è lecito (non sui valori). Le copie heap rispettano il contratto di `emit` (il chiamante può riusare i buffer). L'array è sufficiente per il numero di chiavi distinte atteso su un corpus da laboratorio; l'ordinamento dei gruppi rende deterministica l'assegnazione ai batch.

8. Formato e ordine dell'output

Il file di output è in formato binario ed ha lo stesso layout del messaggio risultato (sezione 6):

- int lunghezza token
- byte effettivi del token (senza `'\0'`)
- int lunghezza risultato

- byte opachi del risultato

Il main accumula i record in un array (`record_from_reducer_t`), poi scrive il file.

I **record** nel file sono in **ordine lessicografico** sul token (`strcmp`). Se il reducer fa più emit per lo **stesso token**, l'ordine relativo nel file è l'**ordine di quelle chiamate**. Il testo chiede un output deterministico a parità di input, callback e attributi, e di definire l'ordine se ci sono più risultati per chiave. Il sort nel main è indipendente dallo scheduling dei worker reducer: token diversi si ordinano per `strcmp`, non per arrivo sulla pipe. Più emit sullo stesso token restano nell'ordine in cui la callback li ha prodotti (un gruppo, un worker): `qsort` non è stabile, quindi ogni record porta un indice di inserimento usato come tie-break.

9. File di log

Il file di log serve per tenere traccia di quelle che sono tutte le fasi significative del programma in esecuzione, molto utile anche per effettuare eventuale debug del codice.

Valore di default e path: `mr.log` nella directory di lavoro. Viene aperto in modalità `O_APPEND`.

Formato riga:

```
[YYYY-MM-DD HH:MM:SS] [processo] [thread_id] [evento] messaggio
```

- *processo*: main, mapper, reducer;
- *thread id*: 0 per main e per i reader; 1..N per i worker;
- *evento* (principali): pipe, fork, `thrd_start`, `thrd_end`, close, output, stats, error;
- *stats*: righe inviate al mapper, coppie emesse, token distinti, record finali.

Esempio: `[2026-06-21 14:30:01] [main] [0] [pipe] created 3 pipes`

Vengono **coperti tutti gli eventi** minimi richiesti: creazione pipe e processi, avvio/fine thread, apertura/chiusura file di input e output, i quattro contatori, errori vari. Viene effettuata una **mutua esclusione inter/intra-processo** con **semaforo POSIX named** per evitare race condition di accesso al file:

1. `sem_open()`: crea o apre un semaforo nel main all'apertura del log
2. ogni `mr_log_write` fa: `sem_wait / writen / sem_post`. Il nome usa il PID del processo che crea il log, così si evita che le istanze concorrenti di programmi diversi condividano lo stesso semaforo.

Tre processi (e più thread nel mapper/reducer) scrivono lo **stesso file**: senza lock le righe si mescolano. Il semaforo named è ereditabile dopo `fork` (stesso mapping)

e copre sia thread sia processi, come previsto dalla spec. `writen` evita righe spezzate da `write` parziali.

10. Test

`make test` compila `libmr.a` se serve, costruisce i sei eseguibili in `tests/` e li lancia in sequenza. Al primo fallimento la batteria si ferma (`exit` diverso da 0). Sono test automatici di unità e di integrazione: mapper e reducer arrivano dai test, non dalla libreria.

Casi del testo coperti oggi: file vuoto, riga vuota e ultima riga senza `'\n'` (`tests/input`); valori opachi con byte nullo interno (`tests/io` sulla coppia, `tests/integration` end-to-end); due `mr_start` identici con output byte-per-byte uguale; path inesistente e secondo `mr_start` sullo stesso handle (`EINVAL`).

Livello	Eseguibile	Cosa verifica
I/O e log	<code>tests/log</code>	formato riga; default <code>mr.log</code> ; 50+50 scritture padre/figlio dopo <code>fork</code>
	<code>tests/input</code>	file con riga vuota e ultima senza <code>'\n'</code> ; directory in ordine lessicografico; file vuoto
	<code>tests/io</code>	<code>readn/writen</code> ; riga/coppia/risultato; coppia vuota o con byte 0; EOF e header/token troncati su <code>mr_read_pair</code> ; <code>mr_validate_len</code>
Stadi isolati	<code>tests/mapper</code>	figlio con <code>dup2</code> : una riga <code>hello</code> → coppia <code><hello, 42></code>
	<code>tests/reducer</code>	tre coppie sullo stesso token → un solo <code>emit</code> con i tre valori
Integrazione	<code>tests/integration</code>	word-count; directory; determinismo; byte opachi; mapper silenzioso; coda 2 e 4+4 thread; input inesistente e one-shot